CSE 2006 - Programming in Java

Course Type: LP Credits: 3

Prepared by
Mr. C. S Rajpoot
School of Computing Science and Engineering
VIT Bhopal University

Unit-1 Java Introduction

Students who complete this course will be able to

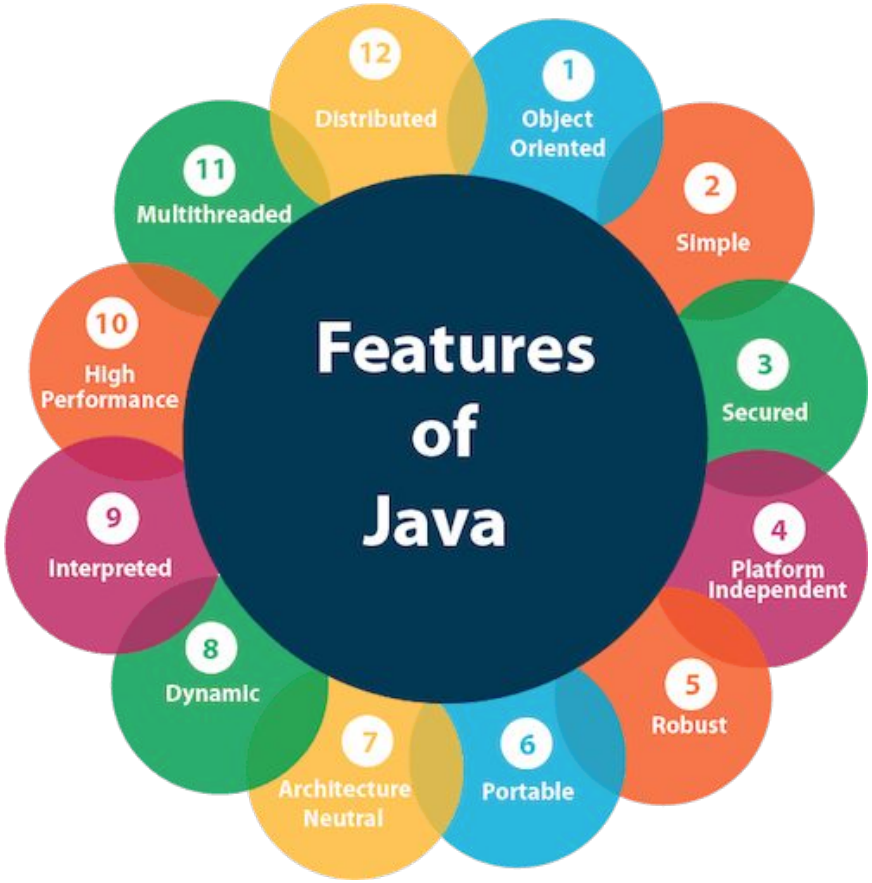
- Analyze the Java flow control using essentials of Java programming
- Solve simple business problems using an object-oriented approach.
- Demonstrate synchronization among different processes using a multithreading approach and exception handling techniques
- Implement Java Input and output streaming using java packages
- Develop and create real-time applications using JDBC and JPA technology

Unit-1 Contents

- **Java Introduction**
- Java Hello World, Java JVM, JRE and JDK, Difference between C & C++, Java Variables, Java Data Types, Java Operators, Java Input and Output, Java Expressions & Blocks, Java Comment
- **Java Flow Control**
- Java if...else, Java switch Statement, Java for Loop, Java for-each Loop, Java while Loop, Java break Statement, Java continue Statement

Features of Java

- The primary objective of **Java programming** language creation was to make it portable, simple and secure programming language. The features of Java are also known as **Java buzzwords.**
- A list of the most important features of the Java language is,
  - **Simple**
  - **Object-Oriented**
  - **Portable**
  - **Platform independent**
  - **Secured**
  - **Robust**
  - **Architecture neutral**
  - **Interpreted**
  - **High Performance**
  - **Multithreaded**
  - **Distributed**
  - **Dynamic**



Features of Java

- **Simple**
- Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun Microsystem, Java language is a simple programming language because:
  - Java syntax is based on C++ (so easier for programmers to learn it after C++).
  - Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
  - There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

Features of Java

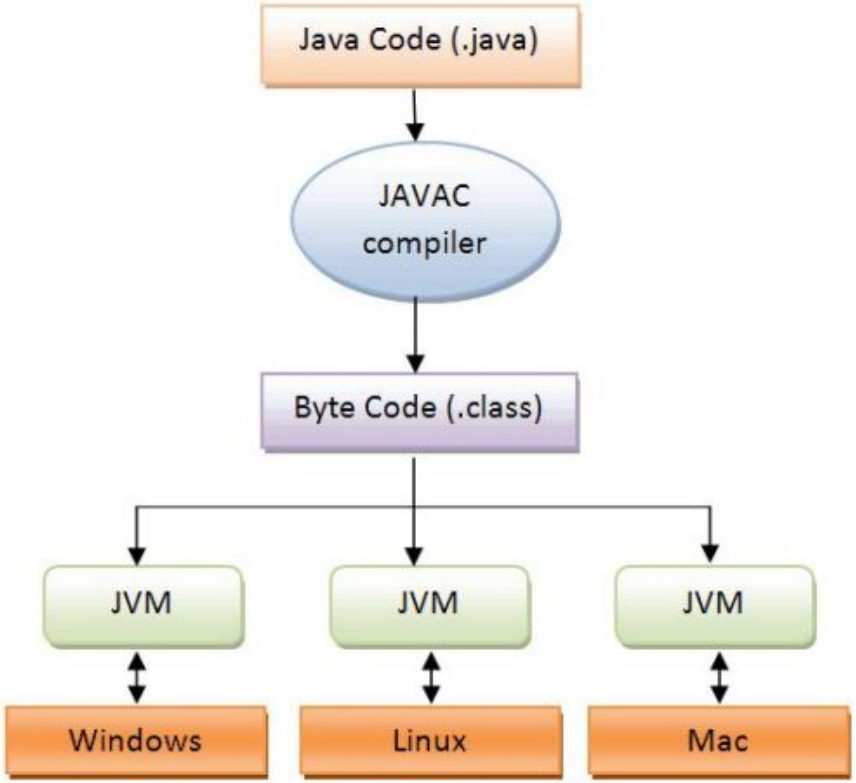
- **Object-oriented**
- Java is an object-oriented programming language. Everything in Java is an **object.** Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior.
- Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.
- **Basic concepts of OOPs are:**
  - Object
  - Class
  - Inheritance
  - Polymorphism
  - Abstraction
  - Encapsulation

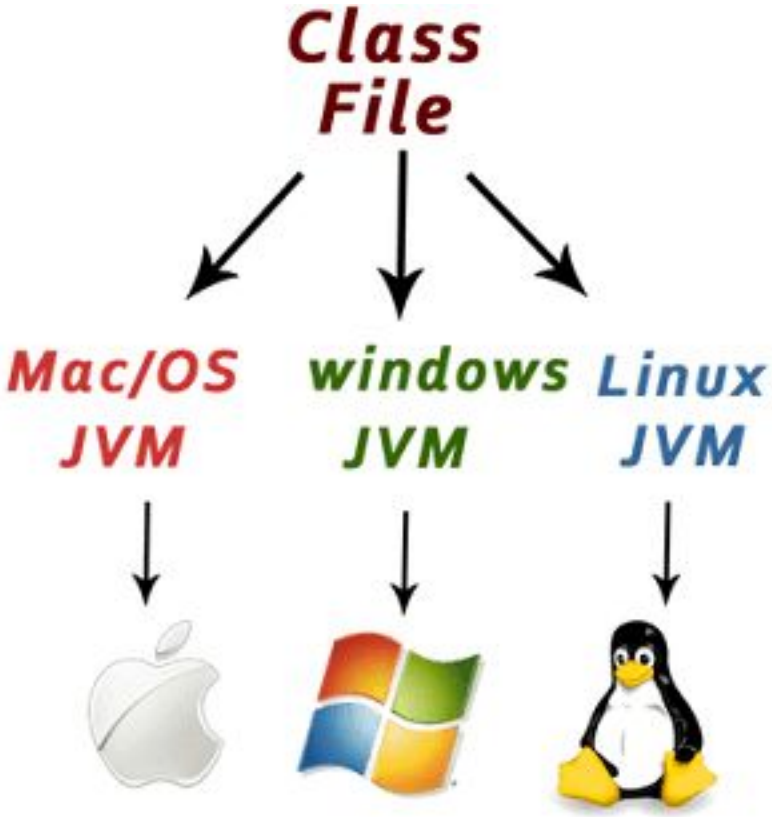
Features of Java

- **Platform independent**
- Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs.
- There are two types of platforms **software-based and hardware-based.** Java provides a software-based platform.
- The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on top of other hardware-based platforms. **It has two components:**
- **Runtime Environment and API(Application Programming Interface)**

Features of Java

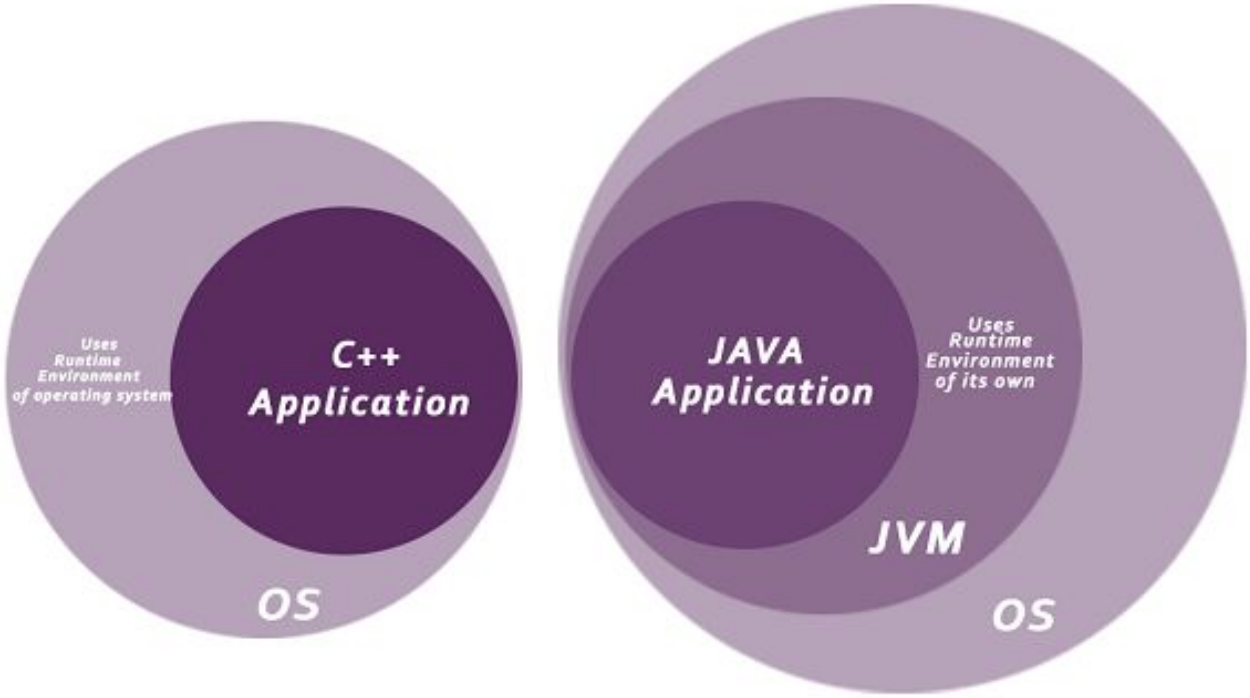
- Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Java code is compiled by the compiler and converted into bytecode. This bytecode is a platform-independent code because it can be run on multiple platforms. i.e.. Write Once and Run Anywhere (WORA).





Features of Java

- **Secured**
- Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:
- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**



Features of Java

- **Secured**
- **Class loader:** Class loader in Java is a part of the Java Runtime Environment (JRE) which is used to load Java classes into the Java Virtual Machine dynamically. It adds security by separating the package for the classes of the local file system from those that are imported from network sources.
- **Byte code Verifier:** It checks the code fragments for illegal code that can violate access rights to objects.
- **Security Manager:** It determines what resources a class can access such as reading and writing to the local disk.
- Java language provides these securities by default. Some security can also be provided by an application developer explicitly through SSL, JAAS, Cryptography, etc.

Features of Java

- **Robust**
- **The English mining of Robust is strong. Java is robust because:**
- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.
- **Architecture-neutral**
- Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.
- In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

Features of Java

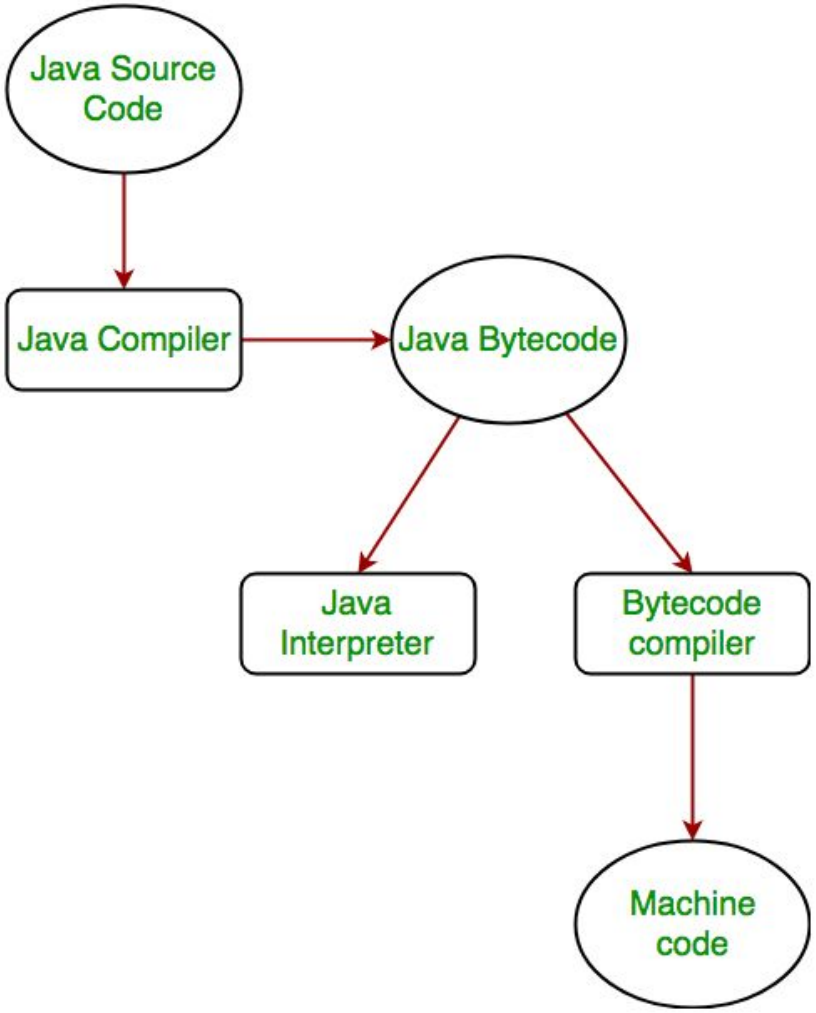
- **Portable**
- Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.
- **High-performance**
- Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.
- **Distributed**
- Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

Features of Java

- **Multi-threaded**
- A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.
- **Dynamic**
- Java is a dynamic language. It supports the dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.
- Java supports dynamic compilation and automatic memory management (**garbage collection**).

Defining how the Java language achieves platform independence

- **Step by step Execution of Java Program:**
- Whenever, a program is written in JAVA, the javac compiles it.
- The result of the JAVA compiler is the **.class file or the bytecode** and not the machine native code (unlike C compiler).
- The bytecode generated is a non-executable code and needs an interpreter to execute on a machine. This interpreter is the JVM and thus the Bytecode is executed by the JVM.
- And finally program runs to give the desired output.



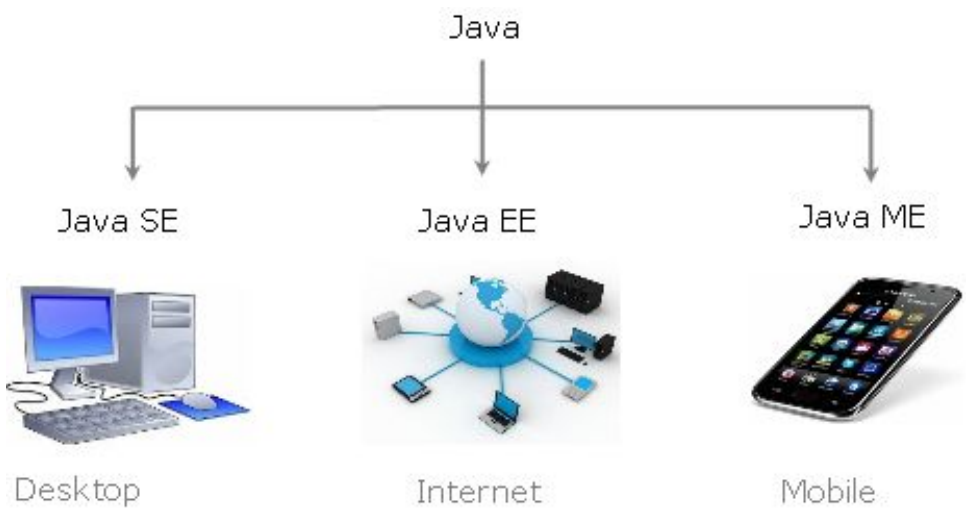
Differentiating between the Java ME, Java SE, and Java EE Platforms

- **4 Edition's of Java**

1. Java Platform, Standard Edition (Java SE)
2. Java Platform, Enterprise Edition (Java EE)
3. Java Platform, Micro Edition (Java ME)
4. Java FX

- **Java SE**
- It defines everything from the basic types and objects of the Java programming language to high-level classes that are used for networking, security, database access, graphical user interface (GUI) development, and XML parsing.
- In addition to the core API, the Java SE platform consists of a virtual machine, development tools, deployment technologies, and other class libraries and toolkits commonly used in Java technology applications.

Differentiating between the Java ME, Java SE, and Java EE Platforms



Differentiating between the Java ME, Java SE, and Java EE Platforms

- **Java EE**
- The Java EE platform is built on top of the Java SE platform. The Java EE platform provides an API and runtime environment for developing and running large-scale, multi-tiered, scalable, reliable, and secure network applications.
- **Java ME**
- The Java ME platform provides an API and a small-footprint virtual machine for running Java programming language applications on small devices, like mobile phones.
- The API is a subset of the Java SE API, along with special class libraries useful for small device application development. Java ME applications are often clients of Java EE platform services.
- **Java FX**
- Java FX technology is a platform for creating rich internet applications written in Java FX Script™. Java FX Script is a statically-typed declarative language that is compiled to Java technology bytecode, which can then be run on a Java VM. Applications written for the Java FX platform can include and link to Java programming language classes, and may be clients of Java EE platform services.

Java Hello World Program

- First Java Program | Hello World Example
- We will learn how to write the simple program of Java. We can write a simple hello Java program easily after installing the JDK.
- To create a simple Java program, you need to create a class that contains the main method. Let's understand the requirement first.
- The requirement for Java Hello World Example
- For executing any Java program, the following software or application must be properly installed.
- Install the JDK if you don't have installed it, download the JDK and install it.
- https://www.oracle.com/java/technologies/downloads/
- Set path of the jdk/bin directory.
- Create the Java program
- Compile and run the Java program

Structure of Java program

Documentation Section

Package Statement

Import Statements

Interface Statements

Class Definitions

main method class

{

main method definition

}

Structure of Java program

Documentation Section

Package Statement

```
import java.util.Scanner;
```

Import Statement

Interface Section

```
public class Demo
```

Class Definition

Opening Brace

```
{
```

Main Method Definition

```
public static void main(String args[])
{
```

Body of Main Function

```
}
```

Closing Brace

```
}
```

Dremendo

Java Hello World Program

- Creating Hello World Example
- Let's create the hello java program:

```
// First java program
class Simple
{
   public static void main(String args[]){
    System.out.println("Hello Java");
   }
}
```

Save the above file as Simple.java.

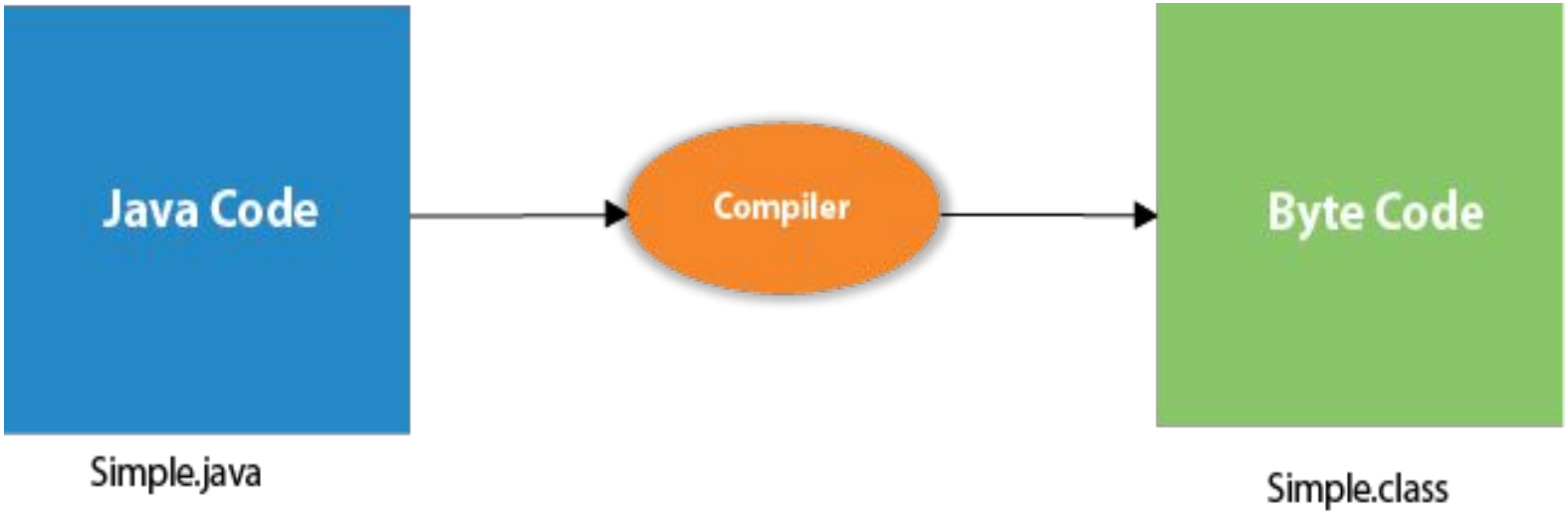
To compile: javac Simple.java

To execute: java Simple

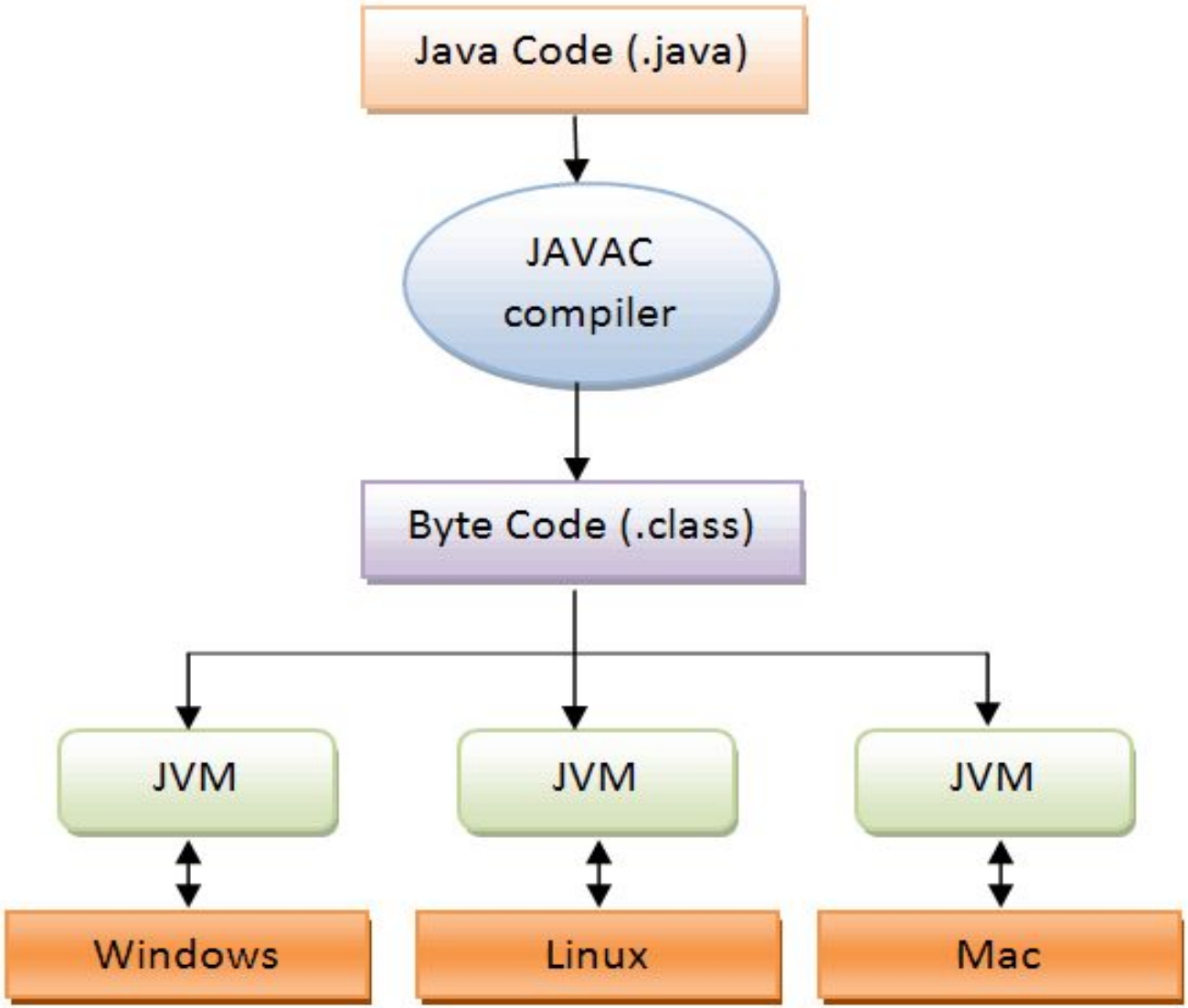
Output:
Hello Java

Compilation Flow

- When we compile Java program using javac tool, the Java compiler converts the source code into byte code.



Compilation Flow



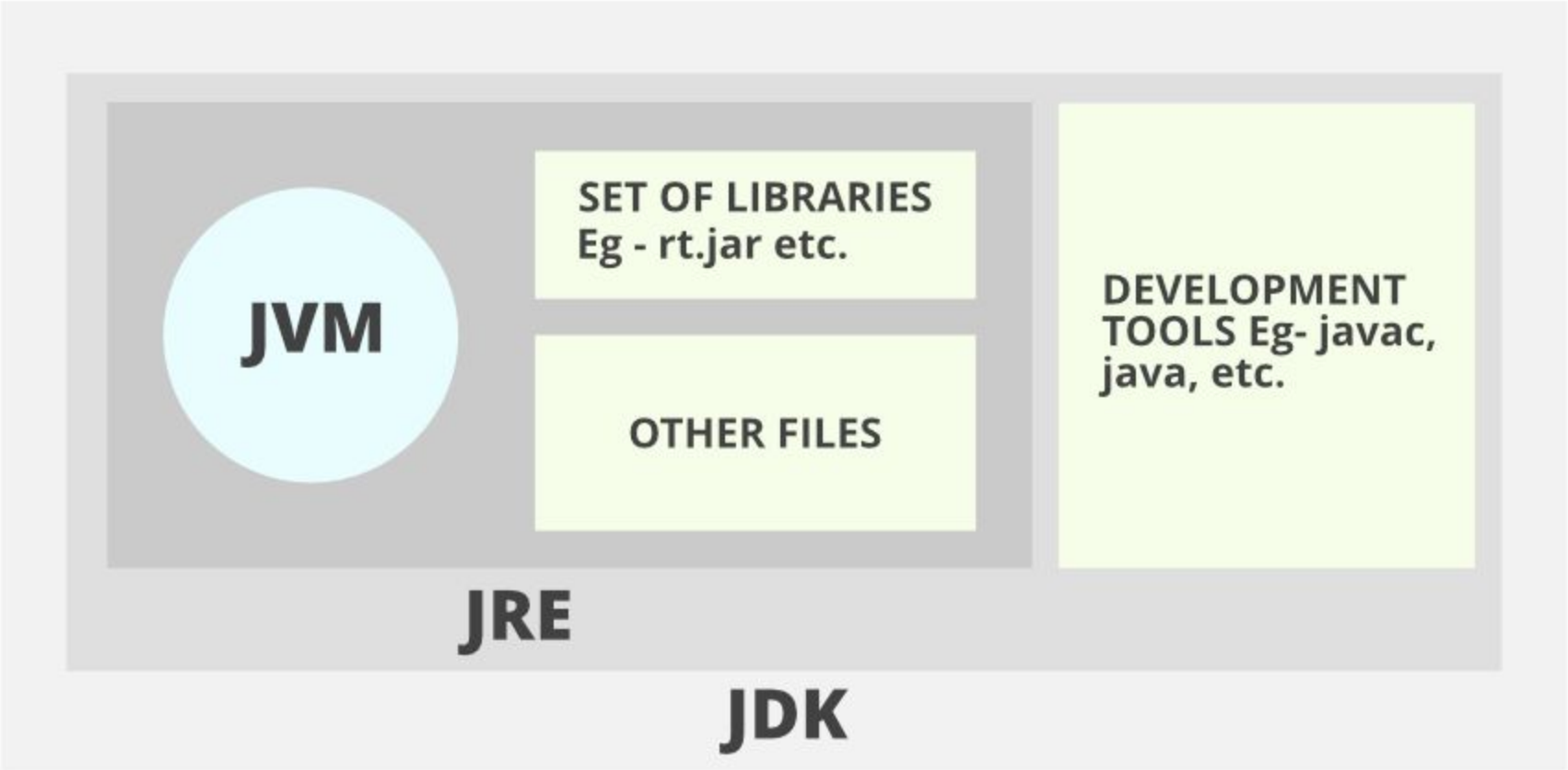
Parameters used in First Java Program

- Let's see what is the meaning of class, public, static, void, main, String[], System.out.println().
- **class** keyword is used to declare a class in Java.
- **public** keyword is an access modifier that represents visibility. It means it is visible to all.
- **static** is a keyword. If we declare any method as static, it is known as the static method. The core advantage of the static method is that there is no need to create an object to invoke the static method. The main() method is executed by the JVM, so it doesn't require creating an object to invoke the main() method. So, it saves memory.
- **void** is the return type of the method. It means it doesn't return any value.
- **main** represents the starting point of the program.
- **String[] args** or **String args[]** is used for command line argument.
- **System.out.println()** is used to print statement. Here, System is a class, out is an object of the PrintStream class, println() is a method of the PrintStream class. We will discuss the internal working of System.out.println() statement in the coming section.

Java JVM, JRE and JDK

- **Java Development Kit** (**JDK**) is a software development environment used for developing Java applications and applets. It includes the Java Runtime Environment (JRE), an interpreter/loader (Java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), and other tools needed in Java development.
- Now we need an environment to make a run of our program. Henceforth, **JRE "Java Runtime Environment"** and may also be written as **"Java RTE."** The Java Runtime Environment provides the minimum requirements for executing a Java application; it consists of the *Java Virtual Machine (JVM), core classes*, and *supporting files*.
- **JVM**, **java virtual machine**. It is as follows:
- A **specification** where the working of Java Virtual Machine is specified. But implementation provider is independent to choose the algorithm. Its implementation has been provided by Sun and other companies.
- An **implementation** is a computer program that meets the requirements of the JVM specification.
- **Runtime Instance** Whenever you write a java command on the command prompt to run the java class, an instance of JVM is created.

Java JVM, JRE and JDK



Java JVM, JRE and JDK

- **1. JDK** (Java Development Kit) is a Kit that provides the environment to **develop and execute(run)** the Java program. JDK is a kit(or package) that includes two things
- Development Tools(to provide an environment to develop your java programs), JRE (to execute your java program).
- **JDK is an implementation of any one of the below given Java Platforms released by Oracle corporation:**
  - Standard Edition Java Platform
  - Enterprise Edition Java Platform
  - Micro Edition Java Platform

2. JRE (Java Runtime Environment) is an installation package that provides an environment to **only run(not develop)** the java program(or application)onto your machine. JRE is only used by those who only want to run Java programs that are end-users of your system.

Java JVM, JRE and JDK

- **3. JVM (Java Virtual Machine)** is a very important part of both JDK and JRE because it is contained or inbuilt in both. Whatever Java program you run using JRE or JDK goes into JVM and JVM is responsible for executing the java program **line by line**, hence it is also known as an ***interpreter*.**
- Now let us discuss the components of JRE in order to understand its importance of it and perceive how it actually works. For this let us discuss components.

Java JVM, JRE and JDK

- **The components of JRE are as follows:**
- **Deployment technologies**, including deployment, Java Web Start, and Java Plug-in.
- **User interface toolkits**, including Abstract Window Toolkit (AWT), Swing, Java 2D, Accessibility, Image I/O, Print Service, Sound, drag, and drop (DnD), and input methods.
- **Integration libraries**, including Interface Definition Language (IDL), Java Database Connectivity (JDBC), Java Naming and Directory Interface (JNDI), Remote Method Invocation (RMI), Remote Method Invocation Over Internet Inter-Orb Protocol (RMI-IIOP), and scripting.
- **Other base libraries**, including international support, input/output (I/O), extension mechanism, Beans, Java Management Extensions (JMX), Java Native Interface (JNI), Math, Networking, Override Mechanism, Security, Serialization, and Java for XML Processing (XML JAXP).
- **Lang and util base libraries**, including lang and util, management, versioning, zip, instrument, reflection, Collections, Concurrency Utilities, Java Archive (JAR), Logging, Preferences API, Ref Objects, and Regular Expressions.
- **Java Virtual Machine (JVM)**, including Java HotSpot Client and Server Virtual Machines.